# 全链路学习每日计划：20 周适配 C++基础

## 一、全链路学习每日计划表（20周，每天2-3小时，适配你的C++基础）

### 整体说明

- 每日任务拆分到「主线学习+实战练习+并行C++任务」，完全贴合之前的全路线
- 每天学习时长控制在2-3小时，任务量适中，不会过载
- 每周末设置复盘日，复习本周内容，补全未完成的任务
- 所有任务都有明确的通关小目标，学完即验证成果

### 第一阶段：编程&数据基础打底 + C++复习（3周，共21天）

#### 阶段总目标：完成基础能力铺垫，C++核心语法复习完成

| 天数 | 主线学习任务（1.5-2小时） | 实战练习（0.5小时） | 并行C++任务（0.5小时，可选，时间不够可放到周末） | 当日通关小目标 |
|---|---|---|---|---|
| Day1 | 廖雪峰Python教程：Python基础、函数基础、高级特性 | 写Python脚本：用列表/字典/集合完成简单的数据统计 | 《C++ Primer》复习：类与对象、构造/析构函数 | 能独立写Python函数，完成数据统计；能看懂C++类代码 |
| Day2 | 廖雪峰Python教程：面向对象编程、模块与包 | 写Python类：封装用户数据的读写操作 | 《C++ Primer》复习：继承与多态、虚函数 | 能写Python的类与继承；能理解C++多态的核心逻辑 |
| Day3 | 廖雪峰Python教程：IO编程、进程线程基础 | 写Python脚本：读写MovieLens CSV文件，做简单过滤 | 《C++ Primer》复习：STL容器（vector、map、unordered_map） | 能用Python读写CSV文件；能用C++ STL容器存储数据 |
| Day4 | 廖雪峰Python教程：数据库操作（psycopg2）、requests网络请求 | 写Python脚本：连接本地PostgreSQL，做简单的CRUD | 《C++ Primer》复习：STL算法、指针与引用 | 能用Python操作PostgreSQL；能用C++ STL算法做数据排序 |
| Day5 | | | | |

| | | | | |
|---|---|---|---|---|
| | 《SQL必知必会》第1-6章：基础SELECT、条件查询、排序分页 | SQLZoo完成SELECT from WORLD、SELECT from Nobel模块 | 《Effective Modern C++》：智能指针（shared_ptr、unique_ptr） | 能写基础的SQL查询；能用C++智能指针管理内存 |
| Day6 | 《SQL必知必会》第7-12章：聚合查询、联表JOIN、子查询 | SQLZoo完成SUM and COUNT、JOIN模块 | 《Effective Modern C++》：lambda表达式、auto类型推导 | 能写SQL联表、聚合查询；能用C++ lambda配合STL算法 |
| Day7 | 【周末复盘】 | 刷LeetCode Python简单题前5道 + SQL简单题前5道 | 用现代C++重写Day1的C++数据统计工具 | 完成刷题任务；C++代码用上智能指针、lambda |
| Day8 | 《SQL必知必会》第13-18章：索引基础、事务基础 | 刷LeetCode SQL简单+中等前10道 | 《Effective Modern C++》：移动语义、右值引用 | 能理解SQL索引和事务；能理解C++移动语义 |
| Day9 | 廖雪峰Git教程：本地仓库操作、add/commit/reset/log | 本地初始化Git仓库，完成3次以上的提交 | 《Effective Modern C++》：范围for循环、std::thread基础 | 能熟练用Git管理本地代码；能写C++简单多线程程序 |
| Day10 | 廖雪峰Git教程：分支管理、merge、冲突解决 | 本地创建dev分支，修改代码，合并到main，解决模拟冲突 | Google C++ Style Guide：命名约定、格式、类规范 | 能熟练用Git做分支管理；C++代码符合Google规范 |
| Day11 | 廖雪峰Git教程：远程仓库操作、clone/push/pull | 注册GitHub账号，创建远程仓库，把本地代码push上去 | 复习本周C++内容，修改之前的代码，符合编码规范 | 能把本地代码提交到GitHub；C++代码规范无问题 |
| Day12 | Python Pandas/NumPy基础：数组操作、DataFrame基础 | 用Pandas读取MovieLens CSV，做简单的过滤、统计 | 用C++写一个完整的用户行为数据统计工具（CSV读写+STL统计） | 能用Pandas做基础的数据处理；C++统计工具能正常运行 |
| Day13 | Python Pandas进阶：groupby、merge、缺失值处理 | 用Pandas统计MovieLens的用户点击次数、电影平均评分 | 给C++统计工具加上多线程统计，提升速度 | 能用Pandas做数据聚合与合并；C++多线程程序无bug |
| Day14 | 【周末复盘】 | 刷LeetCode Python简单题前10道 + SQL简单+中等前20道 | 把所有C++练习代码提交到GitHub，写好README | 完成刷题任务；GitHub仓库有完整的基础练习代码 |

| | | | | |
|---|---|---|---|---|
| Day15-Day21 | 【缓冲周】补全前两周未完成的任务，巩固薄弱点 | 完成第一阶段所有必做实战任务 | 完成C++基础复习的所有实战任务 | 第一阶段所有通关标准全部达成 |

## 第二阶段：后端核心能力搭建 + C++网络编程（6周，共42天）

### 阶段总目标：能独立开发后端接口，熟练使用中间件，C++网络编程入门完成

| 周数 | 核心学习方向 | 每日任务拆分（每天2-3小时） | 通关小目标 |
|---|---|---|---|
| 第4周（Day22-Day28） | Python FastAPI基础 + PostgreSQL ORM | Day22：FastAPI官方文档入门，创建第一个应用，写GET/POST接口<br>Day23：FastAPI路径参数、查询参数、Pydantic模型验证<br>Day24：FastAPI响应处理、错误处理、CORS中间件<br>Day25：SQLAlchemy ORM基础，连接PostgreSQL，创建表<br>Day26：SQLAlchemy CRUD操作，集成到FastAPI<br>Day27：FastAPI依赖注入、接口文档测试<br>Day28：【周末复盘】完成用户管理接口的开发，提交代码到GitHub | 能独立用FastAPI开发用户管理的CRUD接口，能用Swagger测试接口 |
| 第5周（Day29-Day35） | Go语言基础 + Gin框架入门 | Day29：《Go语言圣经》基础语法、结构体、方法<br>Day30：Go接口、错误处理、defer、切片与映射<br>Day31：Go Goroutine、Channel、sync包并发编程<br>Day32：Gin框架入门，创建第一个应用，路由、参数绑定 | 能熟练用Go的并发特性，能用Gin开发简单的RESTful接口 |

| | | | |
|---|---|---|---|
| | | <br>Day33：Gin中间件、响应处理、错误处理<br>Day34：Go go-redis连接Redis，pgx连接PostgreSQL<br>Day35：【周末复盘】用Gin写简单的用户查询接口，提交代码到GitHub | |
| 第6周（Day36-Day42） | Redis核心用法 + Kafka基础 | Day36：Redis常用数据结构（String、Hash、List）命令练习<br>Day37：Redis Set、Sorted Set命令，实现排行榜、限流<br>Day38：Python/Go连接Redis，实现缓存、限流功能<br>Day39：Kafka基础概念、本地安装、Topic创建、生产/消费命令<br>Day40：Python/Go连接Kafka，写生产者/消费者<br>Day41：Kafka用户行为日志采集场景实战<br>Day42：【周末复盘】完成Redis+Kafka的练习代码，提交到GitHub | 能用Redis实现缓存、限流、排行榜；能用Kafka生产/消费消息 |
| 第7周（Day43-Day49） | Elasticsearch基础 + Docker容器化 | Day43：Elasticsearch基础概念、本地安装、Index/Document操作<br>Day44：Elasticsearch基础查询（Match、Term、Bool Query）<br>Day45：Python连接Elasticsearch，实现内容检索<br>Day46：Docker基础、常用命令、镜像拉取/运行<br>Day47：Dockerfile编写，打包FastAPI/Go应用<br>Day48：Docker Compose基础，编写多服务启动配置<br>Day49：【周末复盘】把 | 能用Elasticsearch做简单的内容检索；能用Docker打包部署应用 |

| | | | |
|---|---|---|---|
| | | FastAPI/Go应用打包成Docker镜像，用Docker Compose启动 | |
| 第8周（Day50-Day56） | C++网络编程 + 中间件客户端 | Day50：《UNIX网络编程》Socket基础、TCP套接字编程<br>Day51：C++写TCP服务器/客户端，完成消息收发<br>Day52：C++简单HTTP服务器，返回JSON响应<br>Day53：hiredis连接Redis，实现简单的get/set操作<br>Day54：librdkafka连接Kafka，写生产者/消费者<br>Day55：nlohmann/json解析/生成JSON，集成到HTTP服务器<br>Day56：【周末复盘】完成C++ HTTP服务器+Redis+Kafka的练习，提交代码 | 能用C++写简单的HTTP服务器；能用C++连接Redis和Kafka |
| 第9周（Day57-Day63） | 【缓冲周】补全薄弱点，整合后端服务 | 每天完成一个小模块的开发，把FastAPI、Go、中间件整合起来<br>周末完成所有后端服务的联调，确保所有接口正常 | 第二阶段所有通关标准全部达成，后端服务能正常联调 |

## 第三阶段：AI推荐系统核心能力 + C++模型推理（7周，共49天）

### 阶段总目标：掌握推荐系统全链路，能独立训练/部署推荐模型，C++模型推理入门完成

| 周数 | 核心学习方向 | 每日任务拆分（每天2-3小时） | 通关小目标 |
|---|---|---|---|
| 第10周（Day64-Day70） | 特征工程基础 + 用户/物品特征构建 | Day64：Pandas/NumPy进阶，数据清洗、异常值处理<br>Day65：特征处理：数值特征归一化、类别特征One-Hot编码<br>Day66：时间特征提 | 能独立完成推荐系统的特征工程，构建用户/物品/交互特征 |

| | | | |
|---|---|---|---|
| | | 取、文本特征基础处理<br>Day67：MovieLens用户特征构建（点击次数、平均评分、偏好类型）<br>Day68：MovieLens物品特征构建（点击次数、平均评分、类型编码）<br>Day69：用户-物品交互特征构建，训练集/测试集划分<br>Day70：【周末复盘】完成所有特征构建，保存为数据集，提交代码 | |
| 第11周（Day71-Day77） | 传统推荐算法 | Day71：推荐系统基础概念、全链路、评价指标<br>Day72：基于物品的协同过滤（Item-CF）原理、相似度计算<br>Day73：Python实现Item-CF，用MovieLens数据集测试<br>Day74：基于内容的推荐（CB）原理、实现<br>Day75：基于逻辑回归（LR）的CTR预测原理、scikit-learn实现<br>Day76：三个算法的离线评估（准确率、召回率、AUC）、对比<br>Day77：【周末复盘】完成三个算法的实现，提交代码，写好评估报告 | 能独立实现3种传统推荐算法，能用离线指标评估效果 |
| 第12-13周（Day78-Day91） | 工业级深度学习推荐模型 | Day78：PyTorch基础回顾、张量操作、模型定义<br>Day79：Wide&Deep模型原理、结构拆解<br>Day80：PyTorch实现Wide&Deep，用MovieLens数据集训练<br>Day81：Wide&Deep模型评估、调优<br>Day82：DeepFM模型原理、FM层实现 | 能独立用PyTorch实现Wide&Deep、DeepFM模型，能训练、评估、保存模型 |

| | | | |
|---|---|---|---|
| | | <br>Day83：PyTorch实现DeepFM，训练、评估<br>Day84：DeepFM与Wide&Deep的效果对比<br>Day85：【周末复盘】完成两个模型的实现，提交代码，保存模型文件<br>Day86：DIN模型原理、注意力机制实现<br>Day87：PyTorch实现DIN的注意力层<br>Day88：DIN模型训练、评估<br>Day89：三个深度学习模型的效果对比、优缺点分析<br>Day90：模型保存为PyTorch格式和TorchScript格式<br>Day91：【周末复盘】完成所有模型的实现，提交代码，整理模型文件 | |
| 第14周（Day92-Day98） | 向量数据库Milvus + 大模型+推荐基础 | Day92：向量检索基础、Embedding、相似度计算、ANN搜索<br>Day93：Milvus本地安装（Docker）、Attu可视化工具使用<br>Day94：Milvus常用操作：创建Collection、插入向量、创建索引、搜索<br>Day95：Python连接Milvus，插入电影Embedding，实现相似电影搜索<br>Day96：Go连接Milvus，实现向量召回接口雏形<br>Day97：大模型+推荐的应用场景、LangChain基础<br>Day98：【周末复盘】完成Milvus向量检索的练习，提交代码 | 能熟练使用Milvus做向量检索，能实现简单的向量召回 |
| 第15周（Day99-Day105） | 模型部署 + 效果评估 | Day99：FastAPI封装DeepFM模型，开发精排 | 能把推荐模型部署成接口，能完成召回+精排的全 |

| | | | |
|---|---|---|---|
| | | 接口<br>Day100：精排接口测试、Docker打包<br>Day101：TorchServe部署PyTorch模型（选学）<br>Day102：推荐系统A/B测试基础、实验设计<br>Day103：推荐系统监控指标、离线+在线指标梳理<br>Day104：召回+精排全链路接口联调<br>Day105：【周末复盘】完成模型部署，全链路接口能正常调用 | 链路联调 |
| 第16周（Day106-Day112） | C++模型推理加速 | Day106：LibTorch安装、基础张量操作<br>Day107：C++加载TorchScript格式的DeepFM模型<br>Day108：C++ LibTorch模型推理、输入输出处理<br>Day109：PyTorch模型转ONNX格式<br>Day110：ONNX模型转TensorRT Engine<br>Day111：C++ TensorRT推理、性能对比<br>Day112：【周末复盘】完成LibTorch和TensorRT的推理练习，提交代码 | 能用C++加载推荐模型并做推理，能用TensorRT做模型加速 |
| 第17周（Day113-Day119） | 【缓冲周】补全薄弱点，整合推荐全链路 | 每天完成一个推荐模块的优化，把特征工程、召回、精排、部署整合起来<br>周末完成推荐全链路的联调，确保所有模块正常工作 | 第三阶段所有通关标准全部达成，推荐全链路能正常运行 |

## 第四阶段：全链路项目整合 + 就业冲刺（4周，共28天）

### 阶段总目标：完成全链路项目上线，准备面试，达到入门就业水平

| 周数 | 核心学习方向 | 每日任务拆分（每天2-3小时） | 通关小目标 |
| --- | --- | --- | --- |
| 第18周（Day120-Day126） | 全链路项目整合 | Day120：项目环境搭建，Docker Compose启动所有中间件<br>Day121：数据层开发，数据库表创建、数据导入、特征初始化<br>Day122：Python FastAPI服务开发（用户管理、精排接口、日志上报）<br>Day123：Go Gin服务开发（召回接口、实时日志处理、限流）<br>Day124：算法层整合，特征工程脚本、模型部署接口<br>Day125：全链路接口联调，用户请求推荐的完整流程打通<br>Day126：【周末复盘】修复bug，确保所有功能正常 | 全链路推荐系统项目能在本地正常运行，所有功能正常 |
| 第19周（Day127-Day133） | C++引擎整合 + 项目优化 | Day127：C++高性能召回引擎开发，Milvus C++ SDK集成<br>Day128：C++ HTTP服务器开发，提供召回接口<br>Day129：C++召回引擎Docker打包<br>Day130：C++引擎整合进项目，替换Go召回接口<br>Day131：性能压测，C++与Go版本的QPS/延迟对比<br>Day132：项目优化，bug修复，性能调优<br>Day133：【周末复盘】完成项目文档编写（README、设计文档、总结） | C++召回引擎整合完成，项目文档齐全，代码全部提交到GitHub |
| 第20周（Day134-Day140） | 面试&就业准备 | Day134：知识点梳理，后端+推荐系统+C++核心知识点笔记整理 | 简历优化完成，笔试刷题完成，能流畅讲解项目，能回答高频面试题 |

| | | | |
|---|---|---|---|
| | | <br>Day135：简历优化，用STAR法则写项目经历<br>Day136：笔试刷题，LeetCode算法简单+中等前50道<br>Day137：笔试刷题，LeetCode SQL简单+中等前30道<br>Day138：高频面试题准备，自我介绍、项目讲解、面试题答案<br>Day139：模拟面试，项目讲解、面试题问答练习<br>Day140：【最终复盘】补全所有薄弱点，准备投递简历 | |

# 二、全链路推荐系统项目代码框架（可直接运行，支持扩展）

## 项目总目录结构

Code block

```
recommend-system-fullstack
├── backend-python/        # FastAPI服务：用户管理、精排接口、业务逻辑
├── backend-go/            # Go Gin服务：召回接口、实时日志处理
├── backend-cpp/           # C++高性能召回引擎
├── data/                  # 数据集、初始化脚本、特征工程代码
├── docker-compose.yml     # 一键启动所有服务
└── README.md              # 项目说明文档
```

## 1. backend-python（FastAPI核心代码）

### 目录结构

Code block

```
backend-python
├── app/
│   ├── __init__.py
```

```
4   │   ├── main.py            # 服务入口、路由注册
5   │   ├── models.py          # Pydantic模型、SQLAlchemy ORM模型
6   │   ├── database.py        # 数据库连接
7   │   ├── api/
8   │   │   ├── user.py        # 用户管理接口
9   │   │   ├── recommend.py   # 精排接口
10  │   │   └── log.py         # 行为日志上报接口
11  │   └── utils/
12  │       ├── model_loader.py   # 加载DeepFM模型
13  │       └── feature_loader.py # 加载用户/物品特征
14  ├── models/                # 训练好的DeepFM模型文件
15  ├── requirements.txt       # Python依赖
16  └── Dockerfile             # Docker打包配置
```

## 核心文件示例：app/[main.py](main.py)

Code block

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api import user, recommend, log

app = FastAPI(title="推荐系统后端服务", version="1.0.0")

# 跨域配置
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 路由注册
app.include_router(user.router, prefix="/api/user", tags=["用户管理"])
app.include_router(recommend.router, prefix="/api/recommend", tags=["推荐精排"])
app.include_router(log.router, prefix="/api/log", tags=["行为日志"])

@app.get("/")
def health_check():
    return {"status": "ok", "message": "推荐系统服务运行正常"}
```

## 核心文件示例：app/api/[recommend.py](recommend.py)

Code block

```python
from fastapi import APIRouter, HTTPException
from app.models import RecommendRequest, RecommendResponse
from app.utils.model_loader import deepfm_model
from app.utils.feature_loader import get_user_features, get_item_features
import torch

router = APIRouter()

@router.post("/rank", response_model=RecommendResponse)
def rank_items(request: RecommendRequest):
    """
    精排接口：接收召回的物品ID，用DeepFM模型排序，返回Top-N
    """
    # 获取用户特征
    user_features = get_user_features(request.user_id)
    if not user_features:
        raise HTTPException(status_code=404, detail="用户不存在")

    # 获取物品特征
    item_features_list = get_item_features(request.recall_item_ids)
    if not item_features_list:
        raise HTTPException(status_code=404, detail="物品不存在")

    # 模型推理
    model_input = torch.tensor([user_features + item_features for item_features in item_features_list], dtype=torch.float32)
    with torch.no_grad():
        ctr_scores = deepfm_model(model_input).squeeze().numpy()

    # 按CTR分数排序，取Top-N
    item_score_pairs = list(zip(request.recall_item_ids, ctr_scores))
    item_score_pairs.sort(key=lambda x: x[1], reverse=True)
    top_item_ids = [item_id for item_id, score in item_score_pairs[:request.top_n]]

    return {"user_id": request.user_id, "recommend_item_ids": top_item_ids}
```

## 2. backend-go（Go Gin核心代码）

### 目录结构

Code block

```
backend-go
```

```
├── main.go                # 服务入口
├── api/
│   ├── recall.go          # 召回接口
│   └── log_consumer.go    # Kafka日志消费者
├── config/
│   └── config.go          # 配置文件
├── internal/
│   ├── milvus/            # Milvus连接
│   ├── redis/             # Redis连接
│   └── kafka/             # Kafka连接
├── go.mod                 # Go依赖
└── Dockerfile             # Docker打包配置
```

## 核心文件示例：main.go

Code block

```go
package main

import (
 "backend-go/api"
 "backend-go/internal/milvus"
 "backend-go/internal/redis"
 "backend-go/internal/kafka"
 "github.com/gin-gonic/gin"
)

func main() {
 // 初始化连接
 redis.InitRedis()
 milvus.InitMilvus()
 kafka.InitKafka()

 // 启动Kafka日志消费者
 go api.StartLogConsumer()

 // 初始化Gin服务
 r := gin.Default()

 // 路由注册
 r.GET("/health", func(c *gin.Context) {
 c.JSON(200, gin.H{"status": "ok"})
 })
 recallGroup := r.Group("/api/recall")
 {
 recallGroup.Use(api.RateLimitMiddleware()) // 限流中间件
```

```
 recallGroup.POST("/vector", api.VectorRecall)
 }

 // 启动服务
 r.Run(":8081")
}
```

## 核心文件示例：api/recall.go

Code block

```
package api

import (
 "backend-go/internal/milvus"
 "backend-go/internal/redis"
 "github.com/gin-gonic/gin"
 "net/http"
)

type RecallRequest struct {
 UserID int64 `json:"user_id" binding:"required"`
 TopN   int   `json:"top_n" default:"100"`
}

type RecallResponse struct {
 UserID        int64   `json:"user_id"`
 RecallItemIDs []int64 `json:"recall_item_ids"`
}

// VectorRecall 向量召回接口
func VectorRecall(c *gin.Context) {
 var req RecallRequest
 if err := c.ShouldBindJSON(&req); err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
 return
 }

 // 从Redis获取用户最近点击的物品ID
 latestItemID, err := redis.GetUserLatestClick(req.UserID)
 if err != nil {
 c.JSON(http.StatusNotFound, gin.H{"error": "用户无历史行为"})
 return
 }

 // 从Milvus搜索相似物品
```

```
 recallItemIDs, err := milvus.SearchSimilarItems(latestItemID, req.TopN)
 if err != nil {
 c.JSON(http.StatusInternalServerError, gin.H{"error": "召回失败"})
 return
 }

 c.JSON(http.StatusOK, RecallResponse{
 UserID:        req.UserID,
 RecallItemIDs: recallItemIDs,
 })
}
```

## 3. backend-cpp（C++召回引擎核心代码）

### 目录结构

Code block

```
backend-cpp
├── src/
│   ├── main.cpp          # 服务入口
│   ├── http_server.cpp   # HTTP服务器
│   ├── milvus_client.cpp # Milvus连接
│   └── redis_client.cpp  # Redis连接
├── include/              # 头文件
├── CMakeLists.txt        # CMake编译配置
└── Dockerfile            # Docker打包配置
```

### 核心文件示例：src/main.cpp

Code block

```
#include "http_server.h"
#include "milvus_client.h"
#include "redis_client.h"
#include <iostream>

int main() {
    // 初始化Redis和Milvus连接
    RedisClient redis_client("redis", 6379);
    MilvusClient milvus_client("milvus", 19530);

    if (!redis_client.Connect()) {
```

```
        std::cerr << "Redis连接失败" << std::endl;
        return -1;
    }
    if (!milvus_client.Connect()) {
        std::cerr << "Milvus连接失败" << std::endl;
        return -1;
    }

    // 启动HTTP服务器
    HttpServer server(8082, redis_client, milvus_client);
    server.Start();

    return 0;
}
```

## 4. docker-compose.yml（一键启动所有服务）

Code block

```
version: '3.8'

services:
  # 数据库
  postgres:
    image: postgres:17
    environment:
      POSTGRES_USER: recommend
      POSTGRES_PASSWORD: 123456
      POSTGRES_DB: recommend_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - recommend-network

  # 缓存
  redis:
    image: redis:7
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    networks:
```

```yaml
      - recommend-network

  # 消息队列
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on:
      - zookeeper
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
      - "9092:9092"
    networks:
      - recommend-network

  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    networks:
      - recommend-network

  # 向量数据库
  milvus:
    image: milvusdb/milvus:v2.4.0
    command: ["milvus", "run", "standalone"]
    environment:
      ETCD_ENDPOINTS: etcd:2379
      MINIO_ADDRESS: minio:9000
    ports:
      - "19530:19530"
    depends_on:
      - etcd
      - minio
    networks:
      - recommend-network

  etcd:
    image: quay.io/coreos/etcd:v3.5.5
    command: etcd -advertise-client-urls=http://0.0.0.0:2379 -listen-client-urls http://0.0.0.0:2379 --data-dir /etcd
    volumes:
      - etcd_data:/etcd
    networks:
      - recommend-network
```

```yaml

  minio:
    image: minio/minio:RELEASE.2023-03-20T20-16-28Z
    command: minio server /minio_data
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    volumes:
      - minio_data:/minio_data
    networks:
      - recommend-network

  # 检索引擎
  elasticsearch:
    image: elasticsearch:8.13.0
    environment:
```

## 使用说明

1. 安装Docker和Docker Compose
2. 克隆项目到本地，进入项目根目录
3. 执行 `docker compose up -d` 一键启动所有服务
4. 访问 http://127.0.0.1:8000/docs 查看FastAPI接口文档
5. 访问 http://127.0.0.1:8081 访问Go服务
6. 访问 http://127.0.0.1:8082 访问C++召回引擎

（注：文档部分内容可能由 AI 生成）